

Statistical_Modeling_Report

J.Hastings

2025-06-07

Objective: Build a Lie Detector Model

Purpose:

The goal of this project is to predict which participants will successfully complete the program based on their entrance assessment scores:

- 1 = Successfully completed the program through its duration (completer)
- 0 = Unsuccessfully completed the program prior to expiration (non-completer)

The central question driving this project is:

Can I build a model that beats the baseline accuracy of 59% derived from the Bernoulli distribution of the outcome variable?

Tasks:

- **Cleaning Data:** Remove or correct inaccurate records from the dataset.
- **Formatting Data:** Standardize data formats to ensure consistency.

Columns and Their Acronyms:

- **PWS** — Permanency / Navigating Welfare System
- **DL** — Daily Living
- **SC** — Self Care
- **RC** — Relationships & Communication
- **HMM** — Housing & Money Management
- **WSL** — Work & Study Life
- **CEP** — Career & Education Planning
- **LF** — Looking Forward

```
# Load and format the data
my_data = read.csv("/Users/oakmoreroadinc./Desktop/Data Science /Portfolio Data Science /R_script.R/data.csv")
head(my_data)
```

```
## Youth.ID Gender PWS DL SC RC HMM WSL CEP LF Days_in_Program
## 1 1 M 3.80 4.47 4.41 4.33 3.96 4.10 5.00 5.00 264
## 2 2 M 4.00 4.41 4.53 4.78 3.70 4.40 4.10 4.00 467
## 3 3 M 4.30 4.65 4.29 4.78 3.78 4.60 4.11 4.50 49
## 4 4 M 3.55 2.24 2.71 3.83 2.09 2.70 2.11 3.25 908
## 5 5 F 4.90 4.71 4.76 4.83 3.35 4.65 4.78 5.00 241
## 6 6 M 4.50 4.59 4.65 4.83 4.09 4.35 3.89 4.62 495
## Category Result.Score
## 1 Good 1
```

```
## 2      Good      1
## 3      Bad       0
## 4      Good      1
## 5      Bad       0
## 6      Bad       0
```

```
# Find duplicates: How many duplicated rows exist in the df?
duplicate_rows <- sum(duplicated(my_data))
cat("There are", duplicate_rows, "duplicate rows.\n")
```

```
## There are 0 duplicate rows.
```

```
# Finding missing values for all columns
total_missing_values <- sum(is.na(my_data))
cat("There are", total_missing_values, "missing values in total.\n")
```

```
## There are 0 missing values in total.
```

```
colnames(my_data)
```

```
## [1] "Youth.ID"      "Gender"        "PWS"           "DL"
## [5] "SC"            "RC"            "HMM"           "WSL"
## [9] "CEP"           "LF"            "Days_in_Program" "Category"
## [13] "Result.Score"
```

Checking Data variation

```
assess_cols <- c("SC", "CEP", "RC", "LF", "PWS", "HMM", "DL", "WSL")

summary_stats <- data.frame(
  mean = sapply(my_data[, assess_cols], mean, na.rm = TRUE),
  sd    = sapply(my_data[, assess_cols], sd,    na.rm = TRUE)
)

ro((summary_stats), 2)
```

```
##      mean    sd
## SC  4.40 0.62
## CEP 3.99 0.82
## RC  4.55 0.48
## LF  4.52 0.56
## PWS 4.12 0.74
## HMM 3.85 0.72
## DL  4.39 0.68
## WSL 4.24 0.66
```

```
# Add overall ratio of max/min SD
ratio <- max(summary_stats$sd, na.rm=TRUE) / min(summary_stats$sd, na.rm=TRUE)
cat("Max/Min SD ratio:", round(ratio, 2), "\n")
```

```
## Max/Min SD ratio: 1.71
```

I first, pick the columns that hold the assessment scores (SC, CEP, RC, LF, PWS, HMM, DL, WSL). Next, I calculate the **average (mean)** and **spread (standard deviation)** for each assessment. Then, I found the **largest SD** and the **smallest SD**, and divided them to get a ratio. This ratio tells me how uneven the spreads are. A ratio close to 1 means all assessments vary about the same. A high ratio (like 1.7) means one assessment varies much more than another, so you should standardize before PCA.

```
# making sure target is numeric
my_data$Result.Score <- as.integer(my_data$Result.Score)

# Choose predictors (keep it small with n=22)
assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")

# Build & fit
f <- reformulate(assess_cols, response = "Result.Score")
m <- glm(f, data = my_data, family = binomial())
summary(m)
```

```
##
## Call:
## glm(formula = f, family = binomial(), data = my_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  33.4536    18.5200   1.806  0.0709 .
## PWS          -2.2796     2.2256  -1.024  0.3057
## DL            0.7933     3.8338   0.207  0.8361
## SC          -1.0448     3.5996  -0.290  0.7716
## RC          -8.6021     6.2500  -1.376  0.1687
## HMM           1.4400     2.3165   0.622  0.5342
## WSL           6.6385     4.9175   1.350  0.1770
## CEP          -0.1141     2.2017  -0.052  0.9587
## LF          -3.5557     3.2343  -1.099  0.2716
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 29.767  on 21  degrees of freedom
## Residual deviance: 17.329  on 13  degrees of freedom
## AIC: 35.329
##
## Number of Fisher Scoring iterations: 6
```

```
# Predictions
p_hat <- predict(m, type = "response")
pred_class <- ifelse(p_hat >= 0.5, 1, 0)

# (Optional) quick length sanity checks
length(pred_class); length(my_data$Result.Score)
```

```
## [1] 22
```

```
## [1] 22
```

```
# --- Confusion matrix + accuracy (robust for Rmd)
actual <- model.frame(m)$Result.Score # exact rows used by the model

pred_fac <- factor(pred_class, levels = c(0,1))
act_fac <- factor(actual, levels = c(0,1))

keep <- complete.cases(pred_fac, act_fac)
conf_mat <- table(Predicted = pred_fac[keep], Actual = act_fac[keep])
conf_mat
```

```
##           Actual
## Predicted  0  1
##           0  7  1
##           1  2 12
```

```
acc <- mean(pred_fac[keep] == act_fac[keep])
acc
```

```
## [1] 0.8636364
```

```
# --- LOOCV on the same data the model saw
library(boot)
df_m <- model.frame(m)
cv_res <- cv.glm(df_m, m, K = nrow(df_m)) # leave-one-out
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
cv_err <- cv_res$delta[1]
cv_acc <- 1 - cv_err
cv_err; cv_acc
```

```
## [1] 0.3917709
```

```
## [1] 0.6082291
```

Small-sample Logistic Model without Leakage

Why this strategy? - Small dataset (22 rows). A train/test split wastes data and gives unstable results. - Logistic regression with very few features is a strong, interpretable baseline for binary outcomes. - Leave-One-Out Cross-Validation (LOOCV) uses almost all data to train each time and gives a fair estimate of out-of-sample error.

Why this does not leak data: - The model is fit once on the full data for a quick, descriptive look (coefficients, in-sample confusion matrix). - Generalization is assessed with LOOCV using `boot::cv.glm`. In LOOCV, each run trains on $n - 1$ rows and tests on the 1 row left out, so the test row is never used to fit the model or choose thresholds.

What the metrics mean:

- **Coefficients / p-values** → Show each variable's effect. With just 22 rows, I don't worry about them; nothing will look "significant."
- **Deviance & AIC** → Measure fit. Lower is better, but only meaningful when comparing models.
- **Confusion matrix & Accuracy (86%)** → Show how well the model fits the same data it was trained on. This is usually too optimistic.
- **LOOCV error (~39% → ~60% accuracy)** → A much more realistic measure of how the model will do on new data.

Next Steps: - When applying standardization, fit the scaling only on the training fold and apply it to the hold-out sample during cross-validation. This ensures that no information from the test data leaks into the training process, preserving the integrity of the performance estimate.

```
# --- Step 1: Standardize all predictors
assess_cols <- c("PWS", "DL", "SC", "RC", "HMM", "WSL", "CEP", "LF")
my_std <- scale(my_data[, assess_cols])

# --- Step 2: PCA on standardized data
pca <- prcomp(my_std, center = FALSE, scale. = FALSE)

# Keep enough PCs for 90% variance
s <- summary(pca)
k <- which(cumsum(s$importance["Proportion of Variance", ]) >= 0.90)[1]

# Add PCs back to dataset
pcs <- as.data.frame(pca$x[, 1:k])
colnames(pcs) <- paste0("PC", 1:k)
df <- cbind(my_data, pcs)

# --- Step 3: Fit GAM on PCs
if (k > 1) {
  form <- as.formula(
    paste("Result.Score ~ s(PC1) +", paste(paste0("PC", 2:k), collapse = " + "))
  )
} else {
  form <- as.formula("Result.Score ~ s(PC1)")
}
```

```
# Fit GAM
gam_model <- gam(form, data = df, family = binomial)
summary(gam_model)
```

```
##
## Family: binomial
## Link function: logit
##
## Formula:
## Result.Score ~ s(PC1) + PC2 + PC3
##
## Parametric coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)   21.45     47.91   0.448   0.654
## PC2           71.78     84.59   0.849   0.396
## PC3          -79.80    101.80  -0.784   0.433
##
## Approximate significance of smooth terms:
##             edf Ref.df Chi.sq p-value
## s(PC1) 5.736  5.964    1.1  0.981
##
## R-sq.(adj) =      1    Deviance explained = 99.7%
## UBRE = -0.20147  Scale est. = 1          n = 22
```

```
# --- Step 4: In-sample predictions (diagnostic only)
p_hat <- predict(gam_model, type = "response")
pred_class <- ifelse(p_hat >= 0.5, 1, 0)
table(Predicted = pred_class, Actual = df$Result.Score)
```

```
##           Actual
## Predicted  0  1
##           0  9  0
##           1  0 13
```

```
mean(pred_class == df$Result.Score)
```

```
## [1] 1
```

```
# --- Step 5: LOOCV for real performance
cv_res <- cv.glm(df, gam_model, K = nrow(df)) # leave-one-out
```

```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

```
cv_res$delta # CV error rate (~1 - accuracy)
```

```
## [1] 0.3149692 0.2899055
```

GAM Results Using Principal Components (PCs)

- Coefficients (PC2, PC3): Big but unreliable → not meaningful.
- Smooth on PC1: Model bent a lot, but p-value = 0.98 → no strong evidence of effect.
- Fit stats ($R^2 = 1$, Deviance explained = 99.7%): Model memorized the data (overfit).
- In-sample accuracy = 100%: Looks perfect but not trustworthy.
- LOOCV accuracy 31%: The true performance on new data. This is worse than the logistic model's ~60%.

```
# --- Step 1: Use standardized assessment scores (already done as my_std)
std_df <- as.data.frame(my_std)
std_df$Result.Score <- my_data$Result.Score

# --- Step 2: Composite mean from standardized scores
my_data$composite <- rowMeans(my_std, na.rm = TRUE)

comp_model <- gam(Result.Score ~ s(composite),
                  data = my_data, family = binomial, method = "REML")

# Predictions + confusion matrix + accuracy
p_hat1 <- predict(comp_model, type = "response")
pred1 <- ifelse(p_hat1 >= 0.5, 1, 0)
print(table(Predicted = pred1, Actual = my_data$Result.Score))
```

```
##           Actual
## Predicted  0   1
##           0   5   2
##           1   4  11
```

```
mean(pred1 == my_data$Result.Score)
```

```
## [1] 0.7272727
```

```
# LOOCV
cv_res1 <- cv.glm(my_data, comp_model, K = nrow(my_data))
1 - cv_res1$delta[1] # LOOCV accuracy
```

```
## [1] 0.7500636
```

```
# --- Step 3: Logistic regression index (composite2) using standardized data
logit_model <- glm(Result.Score ~ ., data = std_df, family = binomial)

my_data$composite2 <- predict(logit_model, type = "link")

comp2_model <- gam(Result.Score ~ s(composite2),
                  data = my_data, family = binomial, method = "REML")
summary(comp2_model)
```

```
##
## Family: binomial
## Link function: logit
```

```
##
## Formula:
## Result.Score ~ s(composite2)
##
## Parametric coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.9749    0.6816   1.43   0.153
##
## Approximate significance of smooth terms:
##             edf Ref.df Chi.sq p-value
## s(composite2)  1      1  4.514  0.0336 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.438   Deviance explained = 41.8%
## -REML = 6.8366   Scale est. = 1          n = 22
```

```
# Predictions + confusion matrix + accuracy
p_hat2 <- predict(comp2_model, type = "response")
pred2 <- ifelse(p_hat2 >= 0.5, 1, 0)
print(table(Predicted = pred2, Actual = my_data$Result.Score))
```

```
##           Actual
## Predicted 0  1
##           0  7  1
##           1  2 12
```

```
mean(pred2 == my_data$Result.Score)
```

```
## [1] 0.8636364
```

```
# LOOCV
cv_res2 <- cv.glm(my_data, comp2_model, K = nrow(my_data)) #in sample Accuracy
1 - cv_res2$delta[1]    # LOOCV accuracy
```

```
## [1] 0.827964
```

Simplifying Program Completion Prediction with GAM Composite Features

To streamline the prediction process, I transformed the eight assessment scores into two composite attributes:

1. **Composite Mean** — the simple average of all assessment scores.
2. **Composite2** — a weighted score derived from logistic regression.

I then fit GAM models using these single composite features to test whether they could predict program completion more effectively than using all raw features. The confusion matrix indicates where the model correctly classified completers (1) and non-completers (0), while accuracy reflects the overall proportion of correct predictions.

Among the approaches tested, the **Composite2 model performed best**, achieving an **LOOCV accuracy of approximately 83%**, compared to the composite mean and PCA-based models. This result suggests that a single well-engineered composite feature can improve generalization and simplify the modeling process without sacrificing accuracy.


```

# Prepare new Input
new_input <- data.frame(
  PWS = 4.2, DL = 4.5, SC = 4.0, RC = 4.3,
  HMM = 3.9, WSL = 4.1, CEP = 4.2, LF = 4.0
)
#Standardize the new input the same way as training
means <- attr(scale(my_data[, assess_cols]), "scaled:center")
sds <- attr(scale(my_data[, assess_cols]), "scaled:scale")

new_std <- as.data.frame(scale(new_input, center = means, scale = sds))

#Get the composite2 score from the logistic model
new_input$composite2 <- predict(logit_model, newdata = new_std, type = "link")

#Predict with the gam model
prob <- predict(comp2_model, newdata = new_input, type = "response")
class <- ifelse(prob >= 0.5, 1, 0)

cat("Predicted probability:", prob, "\n")

```

```
## Predicted probability: 0.9880252
```

```
cat("Predicted class:", class, "\n")
```

```
## Predicted class: 1
```

Conclusion: Composite2 GAM Prediction Using the **Composite2 GAM model**, the youth’s assessment produced a **predicted probability of completion of 0.99**, indicating the model is almost certain this participant will finish the program. The **predicted class = 1** confirms the classification as a “completer.”

Interpretation Probability (0.99): The model assigns a very high likelihood of program completion based on the assessment profile.

Class (1): Using the decision threshold of 0.5, the participant is classified as a completer. This demonstrates the model’s ability to process unseen data, calculate the composite2 score, and produce both a probability (confidence) and a classification (decision). Overall, the **Composite2 GAM model** generalized well and delivered interpretable predictions, making it a practical tool for identifying participants most likely to complete the program. Notably, youths who gave themselves perfect scores (all 5’s) were often discharged earlier than expected, suggesting that inflated self-assessments are not reliable indicators of actual progress.

Model Evaluation and Baseline Comparison The **baseline accuracy*** derived from Bernoulli inference was **59%**, representing the performance of a naïve “always complete” prediction. The **Composite2 GAM** model achieved an **LOOCV accuracy of 83%**, significantly outperforming this baseline and showing strong generalization.

In comparison:

- Logistic regression achieved 60% LOOCV accuracy, roughly matching the baseline.
- The PCA-based model overfit the training data and failed to generalize.
- The composite mean GAM improved performance, but the Composite2 model produced the highest accuracy and interpretability.

By leveraging a composite feature derived from logistic regression and applying a GAM, the final model achieved **high predictive power, strong generalization, and clear interpretability**, making it a valuable tool for predicting program completion.